

This style guide is a set of ideas that combined together should make our code maintainable, readable, and should limit the amount of confusion when someone else is looking over your code. This will save countless time when approving code, and helping with other sections of the codebase.

**Idea 1: Code should be explicit.** Comments are great, but they should not be required to understand what is going on with the code base. The code itself should be “self documenting”. Additionally, code should be simple to understand. Avoid deep nested code. Don’t say in comments what can be clearly stated in code.

“Magic Numbers” are any number other than 0 or 1, with the exception of certain special cases, like in printf formatting, and should not be used.

Here’s an example of how you should and should not declare variables, from the C++ core guidelines.

**Example**

```
class Date {  
public:  
    Month month() const; // do  
    int month();         // don't  
    // ...  
};
```

The first declaration of month is explicit about returning a Month and about not modifying the state of the Date object. The second version leaves the reader guessing and opens more possibilities for uncaught bugs.

**Idea 2: Express Intent.** State whether in the comments, or using variable names what the intent of the code is. In pursuit of this, you should not use single letter names, abbreviations, acronyms, and meaningless names as variable names unless you have a good reason to do so.

Make sure to choose descriptive, meaningful names for functions and variables.

Additionally, you should always use explicit declaration of the variable’s datatype unless you have a good, articulable reason not to. Use auto scarcely.

Some good reasons to use auto:

- 1) When the type cannot be explicitly named, such as the type of a lambda expression.
- 2) When the type is difficult to name, particularly when working with templates.
- 3) When the type name is so verbose that writing it explicitly would reduce clarity.

**Idea 3: Don't waste time or space.** When writing code, avoid copying and pasting the same logic into multiple sections. Duplicated code makes an application harder to maintain because changes may need to be made in several places. When logic is reused, place it in an appropriate shared function, class, or other reusable component. Organize related data into structures.

**Naming Convention:**

| Category                         | Naming Convention                    | Example                    |
|----------------------------------|--------------------------------------|----------------------------|
| Variables and parameters         | camelCase                            | targetCount                |
| Functions                        | camelCase                            | activateTarget()           |
| Classes and structures           | PascalCase                           | TargetController           |
| Constants                        | UPPER_SNAKE_CASE                     | MAXIMUM_TARGETS            |
| Private member variables         | camelCase with a trailing underscore | targetCount_               |
| Boolean variables                | camelCase, phrased as a condition    | isConnected, hasError      |
| Variables with measurement units | camelCase, ending with the unit      | durationMs, distanceMeters |

**Formatting:** Use consistent formatting. Use four spaces for each level of indentation, and do not mix tabs and spaces. Place opening braces on the same line as the function, loop, or conditional statement, and place closing braces on their own line. Always use braces for conditional and loop bodies, even when they contain only one statement. Add spaces around binary operators, after commas, and between keywords such as if or while and their opening parentheses. Write one statement per line, and use blank lines to separate logical sections of code. Break long lines at natural points to keep them readable. Apply these conventions consistently throughout the codebase, preferably using a shared automatic formatter configuration.

**Headers and files:** Group related code into appropriately named files, using `.h` for headers and `.cpp` for source files. Place declarations and shared interfaces in headers, and place function definitions in source files, except where definitions need to be visible in headers, such as templates and inline functions. Protect every header against repeated inclusion using include guards. Each header should include the dependencies it needs without relying on another file to include them first. Include the corresponding header first in each source file to help identify missing dependencies. Avoid `using namespace` directives in headers because they affect

every file that includes them. Keep files focused on a clear purpose, and avoid placing unrelated functions into a general-purpose utility file.

**Error Handling:** Check the results of operations that can fail, and validate external input before using it. Use a consistent approach to reporting failures, such as meaningful return values or exceptions. Never silently ignore an error or report success when an operation has failed. Handle errors where the code has enough context to recover, or pass them to the caller when it does not. Error messages should explain what failed and provide useful context without exposing sensitive information. Make sure failures leave the system in a known, safe state, release any resources that are no longer needed, and avoid endless retries.